スポットインスタンス上の短時間ワークロードにおける実行条件の選択による中断対策

那須 渚

2026 年度 修士論文

同志社大学大学院 理工学研究科

概要

スポットインスタンスは計算資源を低価格で利用できる一方，提供側の都合により中断され，ジョブの再実行を要する．CI/CD のビルドやテストのような短時間ワークロードでは，再実行により実行時間と利用コストの双方が増加する．

本論文は，**中断によって失われる時間を最小化する実行条件をどう選ぶか**という問いに取り組む．

まず，短時間ワークロードにおけるスポットインスタンスの信頼性を，起動可能性ではなく**期限条件付き完了可能性** $F_{\mathrm{comp}}(d)$，すなわち起動に成功したインスタンスがワークロード期限 d まで中断されずに利用可能であり続ける割合として定義し，起動成功と起動後の中断挙動を分離して記録する測定フレームワークを実装した．AWS の 4 リージョン・3 インスタンスタイプを対象として 1 週間の実測（$N_{\mathrm{req}} = 2{,}016$）を行い，起動成功率 96.7%，中断率 6.0% を観測した．$F_{\mathrm{comp}}(60)$ はローカル時間帯によって大きく変動し，N. Virginia の計算指向インスタンスでは Night 期間の 0.98 に対し Morning 期間で 0.69 まで低下した．

次に，この実測値を入力として**無駄時間**，すなわち中断による再実行で失われる時間を定式化し，これを最小化する実行条件を選ぶ最適化問題を構築した．無駄時間は，やり直す回数と 1 回あたりの損失の積として表され，いずれも実測値から直接算出できる．

既存データによる評価の結果，実行時刻を選択する戦略は，条件を選ばない場合に対して無駄時間を 49% **削減**し（許容遅延 6 時間のとき），完了率を 0.841 から 0.912 へ改善した．効果は期限 d が長いほど大きく，$d = 60$ 分で最大となる．また期間を分割した検証により，この効果が観測期間内の偶然によるものでないことを確認した．

さらに，リージョンおよびインスタンスタイプまで選択範囲を広げるとより大きな削減が得られるが，これは中断が一件も観測されなかった条件を選ぶことによるものであり，標本数の制約から確定的な結論は得られない．この限界を明示したうえで，実機評価の設計と必要標本数を示す．

目次

概要 1

第 1 章 序論 5
1.1 研究背景 5
1.2 問題意識 5
1.3 本研究の問い 6
1.4 本論文のアプローチ 6
1.5 本論文の貢献 7
1.6 本論文の構成 7

第 2 章 関連研究 8
2.1 スポットインスタンスの可用性と中断対策 8
2.2 中断挙動の時間的特性 8
2.3 チェックポイントによる中断対策 8
2.4 本研究の位置づけ 9

第 3 章 中断特性の実測 10
3.1 本章のねらい 10
3.2 期限条件付き完了可能性 10
3.3 測定フレームワーク 11
3.4 実験設定 12
3.5 実測結果 13
3.6 本章の限界 15
3.7 本章のまとめ 16

第 4 章 実行条件選択のモデル 17
4.1 本章のねらい 17
4.2 コストモデル 17
4.3 比較する戦略 19
4.4 既存データによる評価 19
4.5 考察 23
4.6 本章のまとめ 24

第 5 章　実機評価の設計 25
5.1　検証すべき事項 25
5.2　実験 1：実行時刻選択の効果の実証 25
5.3　実験 2：実行場所の選択 26
5.4　実験 3：中断の独立性の検証 27
5.5　実験規模のまとめ 27
5.6　実装方針 27
5.7　想定される結果 28
5.8　リスクと対策 28

第 6 章　結論 29
6.1　本論文のまとめ 29
6.2　本研究の意義 29
6.3　今後の課題 30

謝辞 31

参考文献 32

参考文献 32

図目次

3.1　測定フレームワーク　. 11
3.2　時間帯，リージョン，インスタンスタイプ別の $F_{\mathrm{comp}}(60)$。注釈は値とサンプル数（$n_{\mathrm{comp}}/n_{\mathrm{succ}}$）を示す。. 13
3.3　経過時間別の中断ハザード率。単調増加の傾向は見られず，0.054–0.130 %/分の範囲で変動する。. 15

4.1　許容遅延（slack）に対する無駄時間。slack が大きいほど選択できる時間帯が増え，無駄時間が減少する。. 20
4.2　N. Virginia，c6a.large における時間帯別の完了可能性（左）と無駄時間（右）。Morning 期間は完了可能性が最も低く，無駄時間が最も大きい。. 20
4.3　期限 d による無駄時間の違い。d が長いほど無駄時間の絶対値が大きく，削減の実効的な意義も大きい。. 21

表目次

2.1 関連研究と本研究の位置づけ 9

3.1 評価対象インスタンスタイプの仕様。観測スポット価格は観測期間中（2026 年 5 月 19 日 –25 日）に 4 リージョンで記録された平均時間単価（USD/h）である。 12
3.2 ローカル時間帯の区分 12
3.3 期限 d ごとの $F_{\mathrm{comp}}(d)$（全データをプール，$N_{\mathrm{succ}} = 1{,}950$） 13
3.4 N. Virginia，c6a.large における時間帯別の $F_{\mathrm{comp}}(60)$ 14
3.5 経過時間別の中断ハザード率（$N_{\mathrm{succ}} = 1{,}950$，中断 117 件） 14
3.6 起動オーバーヘッドの実測値（$n = 1{,}950$） 15

4.1 ジョブの特徴量 17
4.2 比較する戦略 19
4.3 実行時刻の選択による無駄時間の削減（N. Virginia，c6a.large，$d = 60$ 分） . . 20
4.4 期限 d による効果の違い（slack = 6 時間） 21
4.5 期間分割による検証（$d = 60$ 分，slack = 6 時間）。前半 4 日（1,096 件）で条件を選択し，後半 3 日（854 件）で評価した。 22
4.6 選択範囲を広げた場合の無駄時間（$d = 60$ 分，slack = 6 時間） 22

5.1 検証の目的と優先順位 25
5.2 実験 1 の設計 26
5.3 実験 1 で検証すべき予測値（$d = 60$ 分，slack = 6 時間） 26
5.4 実験の規模とコスト 27
5.5 既存実装からの変更点 27
5.6 実験 1 の結果に応じた結論 28
5.7 想定されるリスクと対策 28

第 1 章

序論

1.1 研究背景

近年，クラウドコンピューティングは企業，研究機関，個人開発者によって広く利用されている．クラウド利用の規模が拡大し，高性能な計算資源への需要が増加するにつれて，その利用コストは多くの利用者にとって重要な課題となっている．

コストを削減する手法の一つに，スポットインスタンスの利用がある．スポットインスタンスは，クラウドプロバイダが余剰計算資源をオンデマンドインスタンスより低価格で提供する料金モデルである [1]．オンデマンドインスタンスと比較して，数十パーセントから最大約 90% の割引価格で利用できる [2]．そのため，機械学習やバッチ処理のような大規模計算ワークロードだけでなく，CI/CD における自動ビルドやテストのように一時的に計算資源を必要とするワークロードにも利用されている [3]．

しかしスポットインスタンスは余剰容量に基づいて提供されるため，クラウド需要が増加した場合や利用可能容量が減少した場合に中断されうる．この中断を interruption event と呼ぶ．ワークロードの実行中に interruption event が発生すると，実行中の処理が完了前に停止する．特にバッチ処理，機械学習，CI/CD のビルドやテストが中断された場合，ワークロード全体の再実行が必要となり，実行時間と利用コストの双方が増加する．

1.2 問題意識

既存のアプローチとしては，価格変動を考慮した入札戦略や，Spot Placement Score（SPS）のようなプロバイダ固有の可用性指標に基づく選択が挙げられる [4, 5]．SPS は，スポットリクエストが成立する可能性を推定する上で有用である．

しかし，CI/CD パイプラインやバッチ処理ジョブのような短時間ワークロードの利用者にとって，より実用的な問いは**スポットインスタンスを起動できるかどうか**だけではなく，**ワークロードが完了するまで利用し続けられるかどうか**である．

起動に成功したとしても，ワークロード完了前に中断されれば，短時間ワークロードは失敗する．すなわち**起動時点の可用性**と**起動後の継続性**は異なる問題であり，既存指標は前者のみを扱っている．

1.3　本研究の問い

本論文を貫く問いは一つである．

中断によって失われる時間を最小化するには，ジョブをどの条件で実行すればよいか．

この問いに答えるには，まず「失われる時間」を定量化する必要がある．

中断されるとジョブは再実行される．したがって利用者が支払う総時間は，本来必要な計算時間に加え，中断された試行で費やされた時間が上乗せされる．この上乗せ分を**無駄時間**と呼び，本論文の最小化対象とする．

無駄時間は二つの要素の積として表される．

$$\text{無駄時間} \;=\; \underbrace{\text{やり直す回数}}_{\text{中断されにくさで決まる}} \;\times\; \underbrace{\text{1 回あたりの損失}}_{\text{中断までの経過時間で決まる}} \tag{1.1}$$

ここで重要なのは，**両要素がいずれも実行条件に依存する**点である．

中断されやすさは，リージョン，インスタンスタイプ，および実行する時間帯によって異なりうる．したがって，これらを適切に選べば無駄時間を削減できる可能性がある．

多くの CI/CD ジョブやバッチ処理は，実行先や実行時刻を比較的柔軟に調整できる．しかも実行条件の選択は，**ワークロード自体の改変を必要としない**．チェックポイント機構の実装や処理の分割といったアプリケーション側の対応を伴わずに適用できる点で，実務上の導入障壁が低い．

そこで本論文は，実行条件（リージョン，インスタンスタイプ，実行時刻）を**設計変数**として扱い，無駄時間を最小化する選択を求める問題として定式化する．

1.4　本論文のアプローチ

提案手法を設計するには，まず対象環境における中断の実測データが必要である．式 (1.1) の二要素は，いずれも「どの条件で，どの程度中断されるか」という実測値に基づいて初めて定量化できるためである．

そこで本論文は次の順序で進める．

1. **実測**（第 3 章）：スポットインスタンスの信頼性を期限条件付き完了可能性 $F_{\mathrm{comp}}(d)$ として定義し，測定フレームワークを実装して実測データを取得する．
2. **モデル化と評価**（第 4 章）：実測値を入力として無駄時間を定式化し，これを最小化する実行条件を求める最適化問題を構築する．既存データにより戦略間の比較を行う．
3. **実機評価の設計**（第 5 章）：モデルの予測を実ワークロードで検証する実験を設計する．

1.5　本論文の貢献

本論文の貢献は以下の通りである．

1. 短時間ワークロードにおけるスポットインスタンスの信頼性を，起動可能性やインフラレベルの生存時間ではなく，**期限条件付き完了可能性** $F_{\mathrm{comp}}(d)$ として定義し，起動成功と起動後の中断挙動を分離して記録する測定フレームワークを実装した（第 3 章）．
2. 4 リージョン・3 インスタンスタイプ・1 週間の実測により，$F_{\mathrm{comp}}(d)$ がローカル時間帯によって大きく変動すること（0.69–0.98），および**中断ハザード率が経過時間によらずほぼ一定**であることを示した（第 3 章）．
3. 実測値を入力として**無駄時間を定式化**し，実行条件の選択を最適化問題として定式化した．これにより，従来は観測結果の提示にとどまっていた時間帯依存性を，**実行条件の決定手続き**として利用可能にした（第 4 章）．
4. 既存データによる評価により，実行時刻の選択が無駄時間を 49% **削減**すること，効果が期限 d に依存することを定量的に示した．あわせて期間分割による検証を行い，効果が偶然によるものでないことを確認した（第 4 章）．
5. 標本数の制約により確定できない範囲を明示し，実機評価の設計と必要標本数を示した（第 5 章）．

1.6　本論文の構成

第 2 章では関連研究を整理する．第 3 章では測定フレームワークと実測結果を示す．第 4 章では実行条件選択のモデルを構築し評価する．第 5 章では実機評価の設計を述べる．第 6 章で本論文をまとめる．

第 2 章

関連研究

2.1 スポットインスタンスの可用性と中断対策

スポットインスタンスの可用性および信頼性については，これまでに複数の研究が行われている．Lin ら [2] は可用性を向上させる手法を調査し，チェックポイント，マイグレーション，レプリケーション，スケジューリングといったアプローチを整理している．これらの研究は，スポットインスタンスを実用的に利用するうえで中断リスクが中心的課題であることを示している．

SPS は AWS が提供するプロバイダ固有の指標であり，インスタンスタイプ，リージョン，必要容量などの条件下でスポットリクエストが成立する可能性を推定するために用いられる [7]．Katayama ら [5] はマイグレーション先の選択に SPS を用い，起動時点の可用性を向上させられることを示した．

ただし SPS は，起動に成功したインスタンスがワークロードを完了するのに十分な時間利用可能な状態を維持できるかどうかを直接示すものではない．

2.2 中断挙動の時間的特性

Kim ら [6] は，SPS が時間帯や曜日に応じて周期的に変動し，survival time と一定の関係を持つことを示している．Lee ら [8] は，SPS，スポット価格，中断率などを継続的に収集・保存するアーカイブサービス SpotLake を提案し，GPU インスタンスや大規模インスタンスタイプで可用性が低くなる傾向があることを報告している．

これらの研究は中断挙動の統計的性質を明らかにしているが，時間的変動は主としてスポット市場の特性，すなわち**観測・記述の対象**として扱われている．観測された変動を**実行条件を決定するための入力**として用いる枠組みは十分に検討されていない．

2.3 チェックポイントによる中断対策

中断対策としては，チェックポイントを取得して中断時の損失を抑える手法も広く用いられている [2]．機械学習の分野ではエポック単位でのモデル保存が一般的であり，CI/CD の分野でもテストのシャード分割といった形で事実上の分割実行が行われている．

ただしこれらの手法は，**ワークロード自体の改変を必要とする**．状態の保存と復元を実装できないワークロードには適用できず，また保存コストが大きい場合には利得が失われる．

これに対し本研究が扱う実行条件の選択は，ワークロードに手を加えることなく適用できる．両者は排他的ではなく併用可能であるが，本研究は導入障壁の低い前者に焦点を当てる（§4.5.2 で両者の関係を論じる）．

2.4 本研究の位置づけ

既存研究に対する本研究の位置づけを表 2.1 に示す．

表 2.1 関連研究と本研究の位置づけ

	着目する側面	**時間的変動の扱い**
SPS [7]，Katayama ら [5]	起動時点の可用性	扱わない
Kim ら [6]，Lee ら [8]	中断挙動の統計的性質	観測・記述の対象
チェックポイント手法 [2]	中断時の損失抑制	扱わない（ワークロードの改変を要する）
本研究	**期限条件付き完了可能性**	**実行条件を決定するための入力**

第 3 章

中断特性の実測

3.1 本章のねらい

第 4 章で構築するモデルは，式 (1.1) の二要素を定量化した値を入力として要求する．具体的には，実行条件ごとの

- 期限 d まで中断されずに完了する確率
- 中断された場合に失われる時間の期待値
- インスタンスの起動オーバーヘッド

である．これらはいずれも対象環境における実測値であり，文献値では代用できない．本章では，これらを取得するための測定フレームワークを構築し，AWS 上で実測を行う．

3.2 期限条件付き完了可能性

3.2.1 問題の定義

SPS のような既存指標は，スポットリクエストが成立するかどうかを特徴づける．しかし，起動に成功したインスタンスがワークロード完了まで利用可能であり続けるかどうかは示さない．

CI/CD パイプラインやバッチジョブのような短時間ワークロードにとって，この二つは異なる関心事である．起動に成功しても完了前に中断されれば，ジョブ全体の再実行が必要となり，実行時間とコストの双方が増加する．したがって利用者には，リクエスト成立可能性ではなく**期限指向のスポット信頼性**が必要である．

3.2.2 指標の定義

本研究では実際のアプリケーションレベルのワークロードは実行しない．代わりに，指定期限まで中断されずに利用可能であることを**ワークロード完了可能性の proxy** として用いる．すなわち，起動に成功したインスタンスが期限 d より前に中断されなかった場合，観測条件下においてその実行時間要件を持つ短時間ワークロードは完了可能とみなす．

N_{req} をスポットインスタンスの起動リクエスト数，N_{succ} を起動に成功したインスタンス数，

$N_{\mathrm{int}}(d)$ を期限 d より前に中断された起動成功インスタンス数とする．

起動成功率を次のように定義する．

$$R_{\mathrm{launch}} = \frac{N_{\mathrm{succ}}}{N_{\mathrm{req}}} \tag{3.1}$$

期限前中断率を次のように定義する．

$$R_{\mathrm{int}}(d) = \frac{N_{\mathrm{int}}(d)}{N_{\mathrm{succ}}} \tag{3.2}$$

さらに**期限条件付き完了可能性**を次のように定義する．

$$\boxed{F_{\mathrm{comp}}(d) = 1 - R_{\mathrm{int}}(d)} \tag{3.3}$$

この指標は，起動に成功したインスタンスのうち期限 d まで利用可能な状態を維持した割合を表す．ただしこれは実際のアプリケーションレベルの成功率ではなく，**インスタンスの生存に基づく proxy** として解釈すべきである．

式 (3.1) と式 (3.2) を分離して定義することにより，起動時点の容量不足に起因する失敗と，起動後に発生する中断とを区別して分析できる．

3.3　測定フレームワーク

図 3.1 に測定フレームワークを示す．本フレームワークでは，スケジューラを用いてスポットインスタンスの起動リクエストを定期的に送信し，起動結果，稼働状態，中断状態を収集する．

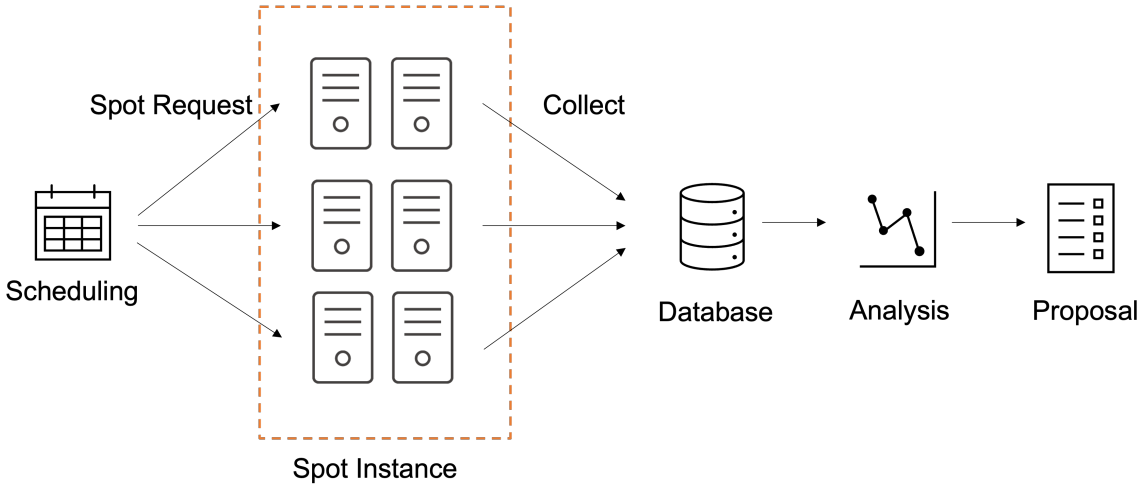


図 3.1　測定フレームワーク

スケジューラは対象とする各インスタンスタイプに対して定期的に起動リクエストを送信する．起動に成功した各インスタンスは 1 時間の固定観測期間にわたって監視される．この期間中，システムはリクエストが成立したか，インスタンスがどの程度の時間利用可能であったか，

予定終了時刻より前に interruption event が発生したかを記録する．観測期間内に中断が発生しなかった場合，インスタンスはシステムによって終了される．

スポット中断と通常のシステム終了を区別するため，測定システムによって終了されることなく予定された 1 時間の観測終了前に利用不能となったインスタンスを中断候補として扱う．

3.4 実験設定

■インスタンスタイプ． 異なるリソース特性を持つインスタンスタイプ間で中断傾向を比較するため，3 種類を代表例として選定した [9, 10]．仕様を表 3.1 に示す．

表 3.1 評価対象インスタンスタイプの仕様。観測スポット価格は観測期間中（2026 年 5 月 19 日 –25 日）に 4 リージョンで記録された平均時間単価（USD/h）である。

	c6a.large	**g4dn.xlarge**	**t3.micro**
分類	計算最適化	GPU 搭載	バースト可能汎用
プロセッサ	AMD EPYC (3rd gen)	Intel Xeon (Cascade Lake)	Intel Xeon (burstable)
vCPU	2	4	2 (burstable)
メモリ	4 GiB	16 GiB	1 GiB
GPU	—	1× NVIDIA T4 (16 GiB)	—
ローカルストレージ	EBS のみ	125 GB NVMe SSD	EBS のみ
観測スポット価格	0.038	0.277	0.005

■リージョン． 対象リージョンは N. Virginia (us-east-1)，Tokyo (ap-northeast-1)，Ireland (eu-west-1)，Singapore (ap-southeast-1) の 4 つである．各リージョンにおいて，各インスタンスタイプにつき 1 時間に 1 台を起動する．

■時間帯の区分． タイムスタンプを各リージョンのローカル時刻に正規化したうえで，ローカル 1 日を 6 時間ずつの 4 区間に分割して記録する（表 3.2）．これにより，各リージョンのローカルな需要サイクルに対して中断傾向を比較できる．

表 3.2 ローカル時間帯の区分

区分	ローカル時刻
Night	00:00–05:59
Morning	06:00–11:59
Afternoon	12:00–17:59
Evening	18:00–23:59

■観測期間と価格設定． 実験は 2026 年 5 月 19 日から 5 月 25 日まで実施した．スポットリクエストでは最大スポット価格を指定していない．したがって本実験は，AWS のデフォルトのス

ポット価格設定における挙動を観測している．

3.5　実測結果

3.5.1　全体の傾向

観測期間中，4 リージョン・3 インスタンスタイプにわたり合計 $N_{\mathrm{req}} = 2{,}016$ 件の起動リクエストを送信した．このうち $N_{\mathrm{succ}} = 1{,}950$ 件が正常に起動し，全体の起動成功率は $R_{\mathrm{launch}} = 96.7\%$ であった．起動に成功したインスタンスのうち 117 件（6.0%）が 1 時間の観測期間終了前に中断された．

期限 d ごとの $F_{\mathrm{comp}}(d)$ を表 3.3 に示す．

表 3.3　期限 d ごとの $F_{\mathrm{comp}}(d)$（全データをプール，$N_{\mathrm{succ}} = 1{,}950$）

期限 d	5 分	10 分	30 分	60 分
$F_{\mathrm{comp}}(d)$	0.9938	0.9882	0.9692	0.9451

3.5.2　実行条件による変動

図 3.2 に，時間帯，リージョン，インスタンスタイプごとに集計した $F_{\mathrm{comp}}(60)$ を示す．

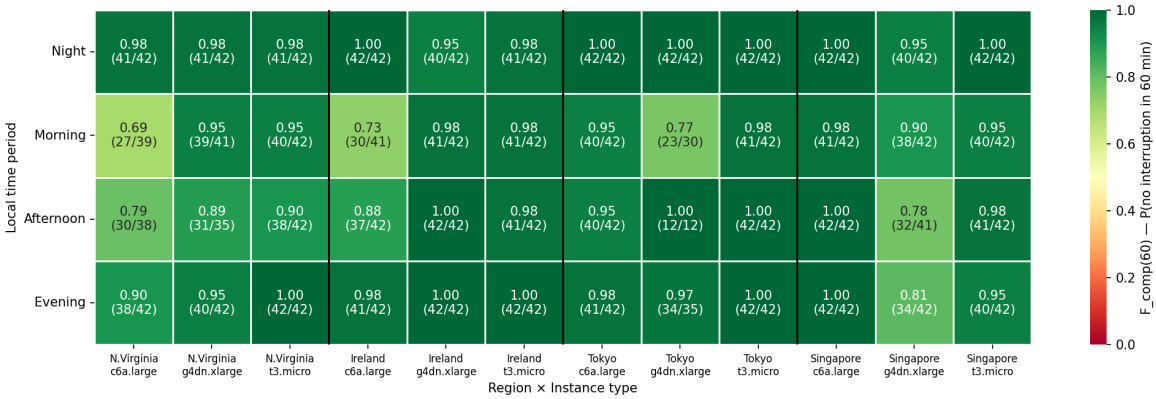


図 3.2　時間帯，リージョン，インスタンスタイプ別の $F_{\mathrm{comp}}(60)$。注釈は値とサンプル数（$n_{\mathrm{comp}}/n_{\mathrm{succ}}$）を示す。

t3.micro はすべての条件において $F_{\mathrm{comp}}(60) \geq 0.95$ を達成しており，軽量インスタンスは中断に対して比較的頑健であることを示している．一方 c6a.large および g4dn.xlarge では条件による変動が大きく，値はおおよそ 0.65 から 1.00 の範囲で変化した．

最大の変動は N. Virginia の c6a.large において観測された．時間帯別の値を表 3.4 に示す．

Night と Morning の差は約 0.29 であり，両者の信頼区間は重ならない．この差が標本変動に起因する可能性は低い．

観測された傾向は，ローカルなクラウド需要が日内周期に従うという仮説と整合する．ローカ

表 3.4　N. Virginia，c6a.large における時間帯別の $F_{\mathrm{comp}}(60)$

時間帯	$F_{\mathrm{comp}}(60)$	95% 信頼区間	n
Night	0.98	0.88–1.00	42
Morning	0.69	0.54–0.81	39
Afternoon	0.79	0.64–0.89	38
Evening	0.91	0.78–0.96	42

ル夜間には業務利用に起因する計算容量への需要が低下し，スポットインスタンスとして利用可能な余剰容量が増加すると考えられる．最もリスクの高い時間帯がリージョンによって異なることもこの解釈を支持する．N. Virginia と Ireland では Morning が最もリスクが高く，これは北米および欧州の業務開始時間帯と対応する．Singapore では Afternoon が最もリスクが高く，アジア地域の午後の利用ピークと整合する．

3.5.3　中断ハザード率の挙動

第 4 章のモデルは，中断が経過時間に対してどのように発生するかに依存する．そこで経過時間区間ごとに，その区間に入る直前まで生存していたインスタンス数（リスク集合）で正規化したハザード率を算出した．結果を表 3.5 および図 3.3 に示す．

表 3.5　経過時間別の中断ハザード率（$N_{\mathrm{succ}} = 1{,}950$，中断 117 件）

経過時間	中断件数	リスク集合	区間あたり	単位時間あたり
0–5 分	12	1,950	0.62%	0.123 %/分
5–10 分	11	1,938	0.57%	0.114 %/分
10–20 分	25	1,927	1.30%	0.130 %/分
20–30 分	12	1,902	0.63%	0.063 %/分
30–40 分	14	1,890	0.74%	0.074 %/分
40–50 分	23	1,876	1.23%	0.123 %/分
50–60 分	10	1,853	0.54%	0.054 %/分

ハザード率は 0.054–0.130 %/分の範囲で変動するが，**経過時間に対する単調増加の傾向は見られない**．すなわち「長時間稼働したインスタンスほど中断されやすくなる」という摩耗的な性質は観測されない．

この性質は第 4 章のモデルにおいて二つの含意を持つ．第一に，中断リスクは稼働時間の経過ではなく**実行条件によって決まる**．したがって条件の選択が有効な対策となる．第二に，長時間のジョブと短時間のジョブで同一の条件選択を適用できる．

■この主張の限界． 中断 117 件という標本数のもとでは，ハザードの定数性を統計的に強く主張することはできない．本論文では「単調増加の証拠が得られなかった」という弱い主張にとどめる．

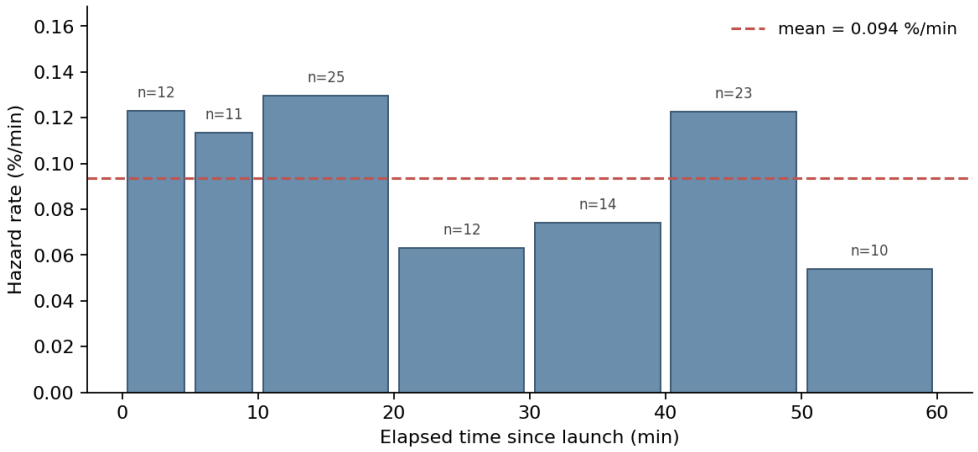


図 3.3　経過時間別の中断ハザード率。単調増加の傾向は見られず，0.054–0.130 %/分の範囲で変動する。

3.5.4　起動オーバーヘッド

インスタンスの起動リクエストから実行開始までに要した時間を表 3.6 に示す．

表 3.6　起動オーバーヘッドの実測値（$n = 1{,}950$）

	平均	中央値	95 パーセンタイル	最大
起動時間	4.52 秒	4 秒	6 秒	30 秒

3.6　本章の限界

本章の測定にはいくつかの限界がある．

- 観測期間は 1 週間に限定されており，各条件のサンプル数は 30–42 である．結果は短期的変動を反映している可能性があり，長期的・季節的傾向として一般化すべきではない．
- 中断されない状態を完了可能性の proxy としており，実際のアプリケーションレベルのワークロードは実行していない．
- 各条件につき 1 時間に 1 台のみ起動しているため，同時起動の影響やキュー挙動は捉えられていない．
- 最大スポット価格を指定していないため，デフォルト価格設定下の挙動のみを反映している．

3.7　本章のまとめ

本章では測定フレームワークを構築し，AWS 上で 1 週間の実測を行った．得られた主要な結果は以下の通りである．

1. 起動成功率 96.7%，中断率 6.0% を観測した．
2. $F_{\mathrm{comp}}(60)$ は実行条件によって 0.69–1.00 の範囲で変動する．最大の差は N. Virginia の c6a.large における Night と Morning の間の 0.29 であり，信頼区間は重ならない．
3. 中断ハザード率は経過時間によらずほぼ一定であり，中断リスクは稼働時間ではなく実行条件によって決まる．
4. 起動オーバーヘッドは平均 4.52 秒である．

結果 2 と結果 3 は，実行条件の選択が中断対策として有効である可能性を示す．次章ではこれを定量的なモデルとして定式化する．

第 4 章

実行条件選択のモデル

4.1　本章のねらい

第 3 章では，$F_{\mathrm{comp}}(d)$ が実行条件によって大きく変動することを示した．しかしこの観測だけでは，**実際にどの条件を選ぶべきか**は決まらない．

たとえば，完了可能性が最も高い条件が最も長い待機を要する場合，それを選ぶべきかは自明でない．また，完了可能性が同程度の条件が複数ある場合，どちらを選んでも同じとは限らない．中断された際に失われる時間が異なりうるからである．

本章では，実測値を入力として**無駄時間**を定式化し，これを最小化する実行条件を求める最適化問題を構築する．これにより，第 3 章の観測結果を**実行条件の決定手続き**として利用可能にする．

4.2　コストモデル

4.2.1　記号と決定変数

ジョブ j は時刻 t_0 に到着し，次で特徴づけられる．

表 4.1　ジョブの特徴量

記号	名称	意味
d	実行所要時間	ジョブの完了に要する時間（分）
D	許容遅延	$t_0 + D$ までに完了すればよいという制約
$\mathcal{R}$	利用可能リージョン	データ所在地制約などにより定まる
$\mathcal{T}$	利用可能タイプ	必要な計算資源により定まる

決定変数は**三つ組** (r, τ, t) である．

$$r \in \mathcal{R}\ (\text{リージョン}),\quad \tau \in \mathcal{T}\ (\text{インスタンスタイプ}),\quad t\ (\text{起動時刻}) \tag{4.1}$$

4.2.2　二要素の定量化

式 (1.1) の二要素を，第 3 章の実測値を用いて書き下す．

■第一要素：やり直す回数． 実行条件 (r, τ, π) のもとで期限 d まで完了する確率を $p(r, \tau, \pi, d)$ とする．ここで $\pi = \pi(r, t)$ は時刻 t におけるリージョン r のローカル時間帯である．この確率は，第 3 章で定義・測定した期限条件付き完了可能性そのものである．

$$p(r, \tau, \pi, d) \;=\; F_{\mathrm{comp}}^{(r,\tau,\pi)}(d) \tag{4.2}$$

成功するまで繰り返すときの試行回数は幾何分布に従い，その期待値は $1/p$ である．したがってやり直す回数は $1/p - 1$ となる．

■第二要素：1 回あたりの損失． 中断された試行で費やされた時間の期待値を

$$\bar{u}(r, \tau, \pi, d) \;=\; \mathbb{E}[\, U \mid U < d \,] \tag{4.3}$$

とする．U は観測された稼働時間であり，第 3 章のデータから直接算出できる．

4.2.3　無駄時間

以上より，無駄時間は次式で与えられる．

$$\boxed{\mathrm{Waste}(r, \tau, t) = \underbrace{\left(\frac{1}{p(r, \tau, \pi(r,t), d)} - 1 \right) \bar{u}(r, \tau, \pi(r,t), d)}_{\text{やり直しによる損失}} + \underbrace{s}_{\text{起動}}} \tag{4.4}$$

ここで s はインスタンス起動オーバーヘッドであり，第 3 章より**実測 4.52 秒**である（表 3.6）．

本モデルの入力はすべて実測値である． p と $\bar{u}$ は第 3 章の観測データから算出され，s も同様に実測されている．プロバイダ固有の指標や推定値を必要としない点で，本モデルは透明かつ再現可能である．

4.2.4　最適化問題

以上より，実行条件の選択は次の最適化問題として記述される．

$$\begin{aligned} \min_{r,\,\tau,\,t} \quad & \mathrm{Waste}(r, \tau, t) && \text{（無駄時間）} \\ \text{s.t.} \quad & t \in [\, t_0,\; t_0 + D - d \,] && \text{（締切）} \\ & r \in \mathcal{R}, \quad \tau \in \mathcal{T} && \text{（利用可能性）} \\ & p(r, \tau, \pi(r,t), d) \geq p_{\min} && \text{（信頼性下限）} \end{aligned} \tag{4.5}$$

■許容遅延と待機可能時間． 制約の第一式は，起動時刻 t を遅らせてよい範囲を定める．D は「いつまでに完了すればよいか」，d は実行所要時間であるから，遅延させてよい上限は $D-d$ となる．これを **slack** と呼ぶ．slack $=0$ のとき $t=t_0$ に退化し，選択できるのはリージョンとタイプのみとなる．

■信頼性下限． 制約の第三式は，完了確率が $p_{\min}$ を下回る条件を除外する．無駄時間の期待値が小さくとも，完了確率が低い条件は実務上受け入れがたいためである．

■求解． 探索空間は，4 リージョン $\times$ 3 タイプ $\times$ 24 時刻 $=288$ 通り程度であり，全列挙により厳密に解ける．

4.3 比較する戦略

式 (4.5) の探索範囲を制限することにより，以下の戦略が定義される．

表 4.2 比較する戦略

戦略	探索範囲	位置づけ
B0: Naive	$t=t_0$，r,τ 固定	条件を選ばない。ベースライン
B1: 時刻選択	t を選択，r,τ 固定	**本章の主たる評価対象**
B2: 条件選択	r,τ を選択，$t=t_0$	待機を要しない
B3: 併用	r,τ,t すべて選択	完全版

B1 と B2 を分離することにより，**実行時刻の寄与**と**実行場所の寄与**を独立に評価できる．

4.4 既存データによる評価

第 3 章のデータを用いて各戦略を評価した．到着時刻 t_0 は 24 時間に一様に分布すると仮定し，各時刻について式 (4.5) を解いて得られる無駄時間を平均した．p と $\bar{u}$ の推定には Wilson 95% 下側信頼限界を併用し，標本数が限られることによる楽観的推定を抑制した．

4.4.1 実行時刻の選択

まずインスタンスタイプを c6a.large，リージョンを N. Virginia に固定し，実行時刻のみを選択する戦略 B1 を評価した．結果を表 4.3 および図 4.1 に示す．

許容遅延が 6 時間のとき，実行時刻の選択により無駄時間は 3.94 分から 2.01 分へ 49% **削減**され，完了確率は 0.841 から 0.912 へ改善した．

slack の増加に伴い削減率は単調に増加するが，その増分は逓減する．これは Night 期間が 6 時間しかないため，slack が一定以上になると最良の時間帯に到達できる到着時刻の割合が飽和するためである．

表 4.3　実行時刻の選択による無駄時間の削減（N. Virginia，c6a.large，$d = 60$ 分）

slack	0 時間	3 時間	6 時間	12 時間
B0（条件を選ばない）	3.94 分	3.94 分	3.94 分	3.94 分
B1（時刻を選択）	3.94 分	2.97 分	**2.01 分**	1.31 分
削減率	—	−24%	**−49%**	−67%
平均完了確率 p	0.841	0.876	**0.912**	0.958

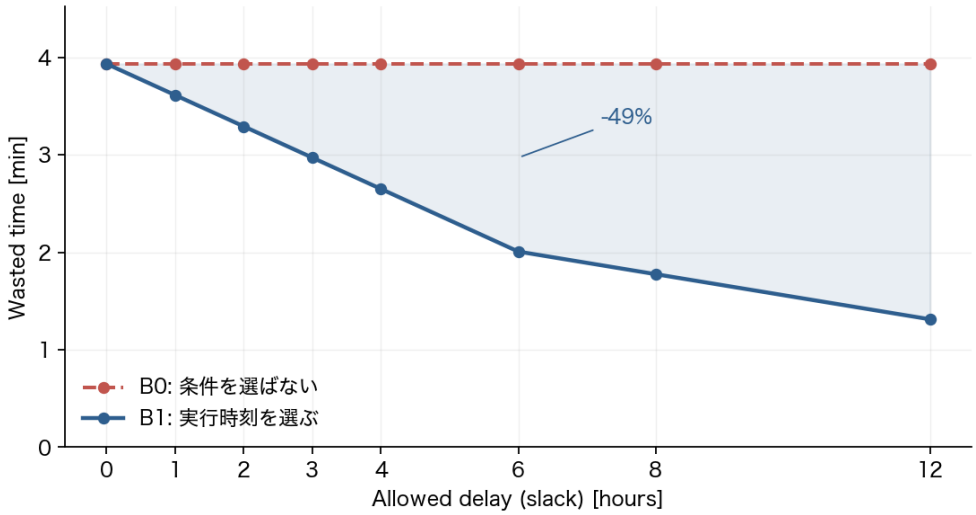


図 4.1　許容遅延（slack）に対する無駄時間。slack が大きいほど選択できる時間帯が増え，無駄時間が減少する。

図 4.2 に，時間帯ごとの完了可能性と無駄時間を示す.

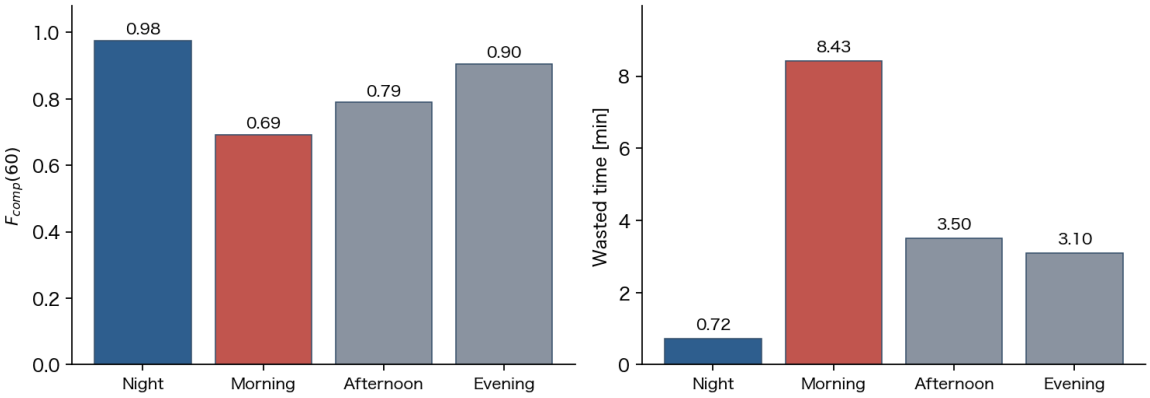


図 4.2　N. Virginia，c6a.large における時間帯別の完了可能性（左）と無駄時間（右）。Morning 期間は完了可能性が最も低く，無駄時間が最も大きい。

4.4.2　期限による効果の違い

期限 d を変化させた場合の結果を表 4.4 および図 4.3 に示す．

表 4.4　期限 d による効果の違い（slack = 6 時間）

期限 d	5 分	10 分	30 分	60 分
B0	0.16 分	0.37 分	1.65 分	3.94 分
B1	0.08 分	0.08 分	0.08 分	2.01 分
削減率	−53%	−80%	−95%	−49%

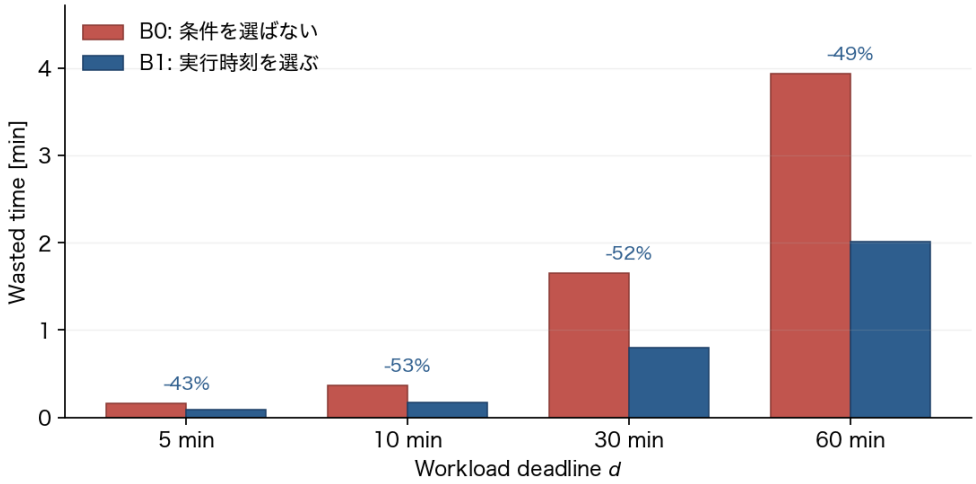


図 4.3　期限 d による無駄時間の違い。d が長いほど無駄時間の絶対値が大きく，削減の実効的な意義も大きい。

削減率は d によって異なるが，**実務上重要なのは削減率ではなく絶対値である**．$d = 5$ 分では無駄時間が 0.16 分であり，53% 削減しても 0.08 分の短縮にすぎない．一方 $d = 60$ 分では 3.94 分から 2.01 分へ 1.93 分の短縮となる．

したがって**実行条件の選択は，実行時間の長いジョブにおいてより大きな意義を持つ**．

4.4.3　効果の頑健性

前節の評価は，条件の選択と効果の測定に同一のデータを用いている．このため，観測期間内で偶然良好であった条件を選び，その偶然を効果として計上している可能性がある．

これを検証するため，観測期間を前半 4 日と後半 3 日に分割し，**前半のデータのみで条件を選択し，後半のデータで評価する**手続きを行った．結果を表 4.5 に示す．

評価期間においても B1 は B0 に対して 30% の削減を示した．選択期間での削減率（60%）より小さいものの，**効果が観測期間内の偶然のみによるものではない**ことが確認された．

表 4.5　期間分割による検証（$d = 60$ 分，slack $= 6$ 時間）。前半 4 日（1,096 件）で条件を選択し，後半 3 日（854 件）で評価した。

戦略	選択期間での値	**評価期間での値**	評価期間での削減
B0	8.39 分	1.34 分	—
B1	3.38 分	**0.94 分**	**−30**%

なお両期間で絶対値が大きく異なるのは，中断の発生が期間によって偏るためである．この変動の大きさ自体が，単一週の観測に基づく推定の不確実性を示している．

4.4.4　実行場所の選択とその限界

リージョンおよびインスタンスタイプまで選択範囲を広げた場合（戦略 B2，B3）の結果を表 4.6 に示す．

表 4.6　選択範囲を広げた場合の無駄時間（$d = 60$ 分，slack $= 6$ 時間）

戦略	選択範囲	無駄時間	削減率
B0	なし	3.94 分	—
B1	時刻	2.01 分	−49%
B2	場所（リージョン・タイプ）	0.10 分	−97%
B3	時刻 + 場所	0.08 分	−98%

B2 および B3 は B1 を大きく上回る削減を示している．しかし**この値はそのまま受け取るべきではない**．

B2 と B3 が選択するのは Tokyo の Night 期間であるが，この条件では観測期間中に**中断が一件も発生していない**（$n = 42$，中断 0 件）．このため完了確率の推定値は 1.00 となり，式 (4.4) の無駄時間はほぼ 0 に評価される．

問題は，48 条件のうち **15 条件（31%）で中断が一件も観測されていない**ことである．中断 117 件が 48 条件に分散するため，$n \approx 40$ の標本では偶然 0 件となる条件が多数生じる．したがって完了確率 1.00 は「中断しない」ことを意味せず，「観測されなかった」ことを意味するにすぎない．

■本論文の立場． 以上より，本論文では **B1（実行時刻の選択）を主たる結果とする**．B1 は同一リージョン内の時間帯比較であり，比較するすべての時間帯で中断が実際に観測されている（表 3.4）．したがって推定値は観測に裏付けられている．

一方 B2 および B3 については，**さらなる削減の余地があることを示唆するにとどめ，確定的な結論は述べない**．これらの確認には標本数の増加が必要であり，第 5 章の実機評価の対象とする．

4.5 考察

4.5.1 なぜ実行条件の選択が有効か

第 3 章で観測されたハザード率の挙動（§3.5.3）が，本章の結果を説明する．

中断ハザード率が経過時間によらずほぼ一定であることは，中断リスクが**稼働時間の長さではなく実行条件によって決まる**ことを意味する．長時間稼働したインスタンスが摩耗的に中断されやすくなるのであれば，条件を選んでも時間の経過とともにリスクが蓄積し，効果は限定的となる．しかし実測ではそのような傾向は見られなかった．

したがって，中断されにくい条件を選べば，その低いリスクが実行期間を通じて維持される．これが実行条件の選択が有効である理由である．

4.5.2 チェックポイントによる対策との関係

中断対策としては，ジョブを分割してチェックポイントを取得し，中断時の損失を抑える手法も広く用いられている．本研究の枠組みにおいて，この手法は式 (1.1) の第二要素（1 回あたりの損失）を削減するものと位置づけられる．

両者は排他的ではなく，併用可能である．しかしチェックポイントによる対策には**ワークロード自体の改変が必要**であり，状態の保存と復元を実装できないワークロードには適用できない．

また予備的な分析によれば，チェックポイントの保存および復元に要する時間が 60 秒を超える場合，分割による利得は失われる．中断率が 6.0% と低いため，中断が発生しない大部分の実行において保存コストが無駄となるためである．

これに対し実行条件の選択は，ワークロードに手を加えることなく適用でき，保存コストを伴わない．本研究がこの手法に焦点を当てる理由である．

4.5.3 実務上の含意

本章のモデルは，実務者に対して次の手続きを与える．

1. 対象環境において $F_{\mathrm{comp}}(d)$ と $\bar{u}$ を測定する（第 3 章の枠組みによる）．
2. ジョブの許容遅延 D と所要時間 d から slack を求める．
3. 式 (4.5) を解き，実行条件を決定する．

この手続きはプロバイダ固有の指標を必要とせず，自身の観測データのみに基づく．また第 3 章で示したとおり，最もリスクの高い時間帯はリージョンによって異なるため，**単一の普遍的な規則を適用するのではなく，対象環境での測定に基づいて決定すべきである**．

4.6　本章のまとめ

本章では，実測値を入力として無駄時間を定式化し，実行条件の選択を最適化問題として構築した．

得られた主要な結果は以下の通りである．

1. 無駄時間は式 (4.4) として定式化され，その入力はすべて第 3 章の実測値から得られる．
2. 実行時刻の選択により，無駄時間は 49% 削減され（slack $=$ 6 時間），完了確率は 0.841 から 0.912 へ改善する．
3. 効果は期限 d が長いほど絶対値として大きく，$d = 60$ 分で 1.93 分の短縮となる．
4. 期間分割による検証により，評価期間においても 30% の削減が確認された．
5. 実行場所まで選択範囲を広げるとさらなる削減が示唆されるが，中断が観測されなかった条件に依存するため，確定的な結論は得られない．

結果 5 が示すとおり，本章の評価は単一週の観測データに基づく限界を持つ．次章では，これを実ワークロードで検証する実験を設計する．

第 5 章

実機評価の設計

5.1　検証すべき事項

第 4 章の評価は，第 3 章で取得した観測データに基づくシミュレーションである．実際のワークロードを実行したものではない．

そこで本章では，モデルの予測を実機で検証する実験を設計する．検証すべき事項を優先順位とともに表 5.1 に示す．

表 5.1　検証の目的と優先順位

優先	目的	理由
1	**B1 の効果の実証**	本論文の主たる主張。予測値（無駄時間 -49%，完了確率 $+7$ ポイント）が実ワークロードで再現されるか
2	モデルの予測精度	式 (4.4) による予測値と実測値の乖離
3	B2・B3 の確認	§4.4.4 で確定できなかった，実行場所の選択による追加的な削減
4	中断の独立性	期待試行回数 $1/p$ の前提の検証

5.2　実験 1：実行時刻選択の効果の実証

■目的． 戦略 B1 の効果を実ワークロードで検証する．

■設計． 対照群と処理群を並行して実行する．

リージョンとタイプを固定するのは，§4.4.4 で述べたとおり，実行場所の選択が標本数の制約を受けるためである．本実験では**実行時刻の効果のみを分離して検証する**．

■測定項目．

- 完了率：期限 60 分までに中断されなかった割合
- やり直し回数
- 無駄時間：中断された試行で費やされた時間の合計

表 5.2　実験 1 の設計

項目	内容
対照群（B0）	到着時刻にそのまま実行
処理群（B1）	式 (4.5) の解に従い，許容遅延 6 時間以内で無駄時間が最小となる時刻に実行
リージョン	N. Virginia（固定）
インスタンスタイプ	c6a.large（固定）
ワークロード	60 分の実ジョブ。チェックポイント機構を持たないもの
到着時刻	毎時 0 分（1 日 24 回）

- 完了までの実時間

■予測値． 第 4 章のモデルによる予測を表 5.3 に示す．

表 5.3　実験 1 で検証すべき予測値（$d = 60$ 分，slack $= 6$ 時間）

指標	B0	B1	差
完了確率	0.841	0.912	+7.1 ポイント
無駄時間	3.94 分	2.01 分	−49%
やり直し率	15.9%	8.8%	−7.1 ポイント

■必要標本数． 完了確率の差（7.1 ポイント）を有意水準 5%，検出力 80% で検出するために必要な標本数は，両群合計で約 690 件である[*1]．1 日 24 回・両群で 48 件を実行する場合，**約 15 日**で必要な標本数に達する．

■コスト． c6a.large の観測スポット価格（0.0295 USD/h）に基づくと，690 件 × 1 時間 × 0.0295 USD ≈ **$20** である．

5.3　実験 2：実行場所の選択

■目的． §4.4.4 で確定できなかった B2・B3 の効果を検証する．

■設計． 実験 1 と並行し，4 リージョンで同一のワークロードを実行する．とりわけ，中断が観測されなかった条件（Tokyo の Night 期間など）について標本を追加する．

■規模． 各条件の標本数を 100 件以上とするには，4 リージョン × 4 時間帯 × 100 件 = 1,600 件の実行を要する．1 日 96 件（4 リージョン × 24 回）で約 17 日である．コストは約 $47 である．

[*1] 二標本比率の検定における標本サイズの近似式 $n \approx 16\,\bar{p}(1-\bar{p})/\delta^2$（片群あたり）による。$\bar{p} = 0.877$, $\delta = 0.071$ として片群約 344 件，両群計約 687 件。正確な式によれば両群計約 671 件であり，以下では安全側をとって 690 件とする。

5.4　実験 3：中断の独立性の検証

■目的．「中断された直後の再試行も同じ確率 p で成功する」という仮定を検証する．

式 (4.4) における期待試行回数 $1/p$ は，各試行が独立であることを前提としている．しかし中断は容量逼迫時に発生すると考えられるため，直後の再試行はより失敗しやすい可能性がある．その場合 p は過大評価であり，本論文の削減率も割り引いて解釈する必要がある．

■方法． 中断を検知した場合に即座に同一条件で再起動し，2 回目の成否を記録する．1 回目の p と 2 回目の p を比較する．

■規模． 実験 1 および 2 に相乗りできる．中断された実行について再試行の結果を記録すればよく，追加のコストを要しない．中断率 6.0% のもとでは，実験 1 と 2 を合わせた約 2,290 件から約 137 件の中断が得られる見込みである．

5.5　実験規模のまとめ

表 5.4　実験の規模とコスト

実験	期間	コスト	得られるもの
1: B1 の実証	約 15 日	\$20	**本論文の主張の検証**
2: B2・B3 の確認	約 17 日	\$47	実行場所の選択の効果
3: 独立性	実験 1・2 に相乗り	\$0	モデルの前提の検証
合計	**約 3 週間**	**約 \$67**	

実験 1 と 2 は並行実行できるため，全体で約 3 週間を見込む．

5.6　実装方針

第 3 章で構築した測定フレームワークを流用する．変更点を表 5.5 に示す．

表 5.5　既存実装からの変更点

既存要素	変更内容
起動処理	待機処理を実ワークロードに置換
中断検知	そのまま流用（インスタンスメタデータの監視）
スケジューラ	戦略に従って起動時刻を決定する処理を追加。待機中は起動予約のみを登録し，計算資源を占有しない
記録項目	戦略名，到着時刻，実際の起動時刻，再試行回数を追加

5.7　想定される結果

実験 1 の結果に応じて，導かれる結論を表 5.6 に示す．**いずれの場合も報告価値のある結果となる．**

表 5.6　実験 1 の結果に応じた結論

実測結果	導かれる結論
予測と整合	実行条件の選択が有効であり，式 (4.4) が実環境を説明できることが示される
効果はあるが予測より小さい	効果の存在は確認されるが，モデルの前提（とりわけ試行の独立性）に修正が必要である
効果が認められない	観測期間と実験期間の間でスポット市場が変化したか，単一週の観測に基づく推定が一般化できないことを示す。**観測の有効期間という新たな問いが生じる**

5.8　リスクと対策

表 5.7　想定されるリスクと対策

リスク	影響	対策
中断が十分に集まらない	検出力の不足	中断率が最も高い条件（N. Virginia，Morning）を対照群の実行対象に含める
観測期間と実験期間の市場変化	予測が外れる	実験期間中に F_{comp} を再測定し，第 3 章の値と比較する
独立性が成立しない	p が過大評価となる	実験 3 で検証し，必要なら条件付き確率へ修正する
実ワークロードの失敗が混入	中断と区別できない	ジョブの終了コードと中断通知を分けて記録する

第 6 章

結論

6.1 本論文のまとめ

本論文は，**中断によって失われる時間を最小化するにはジョブをどの条件で実行すればよいか**という問いに取り組んだ．

第 3 章では，スポットインスタンスの信頼性を期限条件付き完了可能性 $F_{\mathrm{comp}}(d)$ として定義し，起動成功と起動後の中断挙動を分離して記録する測定フレームワークを実装した．AWS の 4 リージョン・3 インスタンスタイプを対象として 1 週間の実測（$N_{\mathrm{req}} = 2{,}016$）を行い，起動成功率 96.7%，中断率 6.0% を観測した．$F_{\mathrm{comp}}(60)$ は実行条件によって 0.69–1.00 の範囲で変動し，最大の差は N. Virginia の c6a.large における Night 期間と Morning 期間の間の 0.29 であった．また中断ハザード率は経過時間によらずほぼ一定であり，中断リスクが稼働時間ではなく実行条件によって決まることを示した．

第 4 章では，これらの実測値を入力として**無駄時間**を定式化し，実行条件の選択を最適化問題として構築した．本モデルの入力はすべて実測値であり，プロバイダ固有の指標を必要としない．既存データによる評価の結果，実行時刻の選択により無駄時間は 49% **削減**され（許容遅延 6 時間のとき），完了確率は 0.841 から 0.912 へ改善した．また期間を分割した検証により，評価期間においても 30% の削減が確認され，効果が観測期間内の偶然のみによるものでないことを示した．

一方，実行場所まで選択範囲を広げた場合にはより大きな削減が示唆されたが，これは中断が一件も観測されなかった条件に依存するものであり，標本数の制約から確定的な結論は得られなかった．この限界を明示したうえで，第 5 章では実機評価の設計と必要標本数（両群計約 690 件，約 3 週間，約 $67）を示した．

6.2 本研究の意義

本研究の意義は以下の三点である．

1. スポットインスタンスの信頼性を，プロバイダ固有の不透明な指標ではなく，直接観測された起動・中断記録のみから算出される**解釈可能な指標**として定式化した．

2. 従来は観測・記述の対象として扱われてきた中断挙動の時間的変動を，**実行条件を決定するための入力**として利用する枠組みを与えた．これにより，観測結果が実行条件の決定手続きとして利用可能になる．
3. 提案手法は**ワークロード自体の改変を必要としない**．チェックポイント機構の実装や処理の分割を伴わずに適用できる点で，実務上の導入障壁が低い．

6.3 今後の課題

- 第 5 章の実機評価の実施
- 観測期間の延長による，季節変動を含む長期的傾向の把握，および観測結果の有効期間の検討
- 実行場所の選択（B2・B3）の効果の確定
- 中断の独立性に関する検証と，成立しない場合のモデルの修正
- チェックポイントによる対策との併用効果の検討
- SPS など既存のプロバイダ固有指標との定量的比較

謝辞

本研究を進めるにあたり，終始丁寧なご指導を賜りました指導教員の先生方に深く感謝いたします．また，日頃より有益な議論をいただいた研究室の皆様に御礼申し上げます．

参考文献

[1] Amazon Web Services: Amazon EC2 Spot Instances. https://aws.amazon.com/ec2/spot/ (Accessed: May 2026)

[2] Lin, L., Pan, L., Liu, S.: Methods for improving the availability of spot instances: A survey. Computers in Industry, 141, 103718 (2022).

[3] Ramesh, D.G.A., Lennon, R.G.: Evaluating the Impact of Reserved Instances vs. Spot Instances in DevOps Cloud Workflows. In: 2025 35th Irish Signals and Systems Conference (ISSC). IEEE (2025).

[4] Garg, D., Kaata, S.K., Bhumireddy, D.R., Bolla, B.K.: Reducing Cloud Cost Through Optimized Bidding for AWS Spot Instances. In: 2025 3rd International Conference on Advances in Computation, Communication and Information Technology (ICAICCIT). IEEE (2025).

[5] Katayama, D., Kasai, K., Koita, T.: Migration Destination Selection Algorithm for Spot Instances using SPS. In: 2022 IEEE International Conference on Big Data (Big Data), pp. 6690–6692. IEEE, Osaka (2022).

[6] Kim, K., Lee, K.: Making Cloud Spot Instance Interruption Events Visible. In: Proceedings of the ACM Web Conference 2024 (WWW '24), pp. 2998–3009. ACM, New York (2024).

[7] Amazon Web Services: Spot Placement Score. Amazon EC2 User Guide. https://docs.aws.amazon.com/AWSEC2/latest/UserGuide/spot-placement-score.html (Accessed: May 2026)

[8] Lee, S., Hwang, J., Lee, K.: SpotLake: Diverse Spot Instance Dataset Archive Service. In: 2022 IEEE International Symposium on Workload Characterization (IISWC), pp. 242–255. IEEE (2022).

[9] Pham, T.-P., Ristov, S., Fahringer, T.: Performance and Behavior Characterization of Amazon EC2 Spot Instances. In: 2018 IEEE 11th International Conference on Cloud Computing (CLOUD). IEEE (2018).

[10] Amazon Web Services: Amazon EC2 Instance Types. Amazon EC2 User Guide. https://docs.aws.amazon.com/AWSEC2/latest/UserGuide/instance-types.html (Accessed: May 2026)